



Advanced Visual Computing (Digital Design)

Lab 5 – meshing, curvature, smoothing, spectral analysis

Andrea Giachetti University of Verona, Italy

andrea.giachetti@univr.it

Today

- Meshing
- Curvatures, smoothing
- Spectral analysis
- Final homework assignment



Open 3D functions

- Open3D features implementations of most of the methods we described in theory
 - Alpha shapes
 - Ball Pivoting
 - Poisson Meshing
 - https://www.open3d.org/docs/latest/tutorial/Advanced/surface_reconstruction.html
- We can test them, varying parameters and evaluating the resulting mesh quality
- Note: Poisson Meshing requires normal estimation
- See `meshing_open3d.py`

Mesh properties



- We can check
 - `print(rec_mesh.is_watertight())`
 - `print(rec_mesh.is_self_intersecting())`
 - `print(rec_mesh.is_edge_manifold())`
 - ...
- Functions to fix issues available
 - https://www.open3d.org/docs/latest/python_api/open3d.geometry.TriangleMesh.html
- Functions to perform mesh decimation or subdivision are available
 - You can test with your own scans/models
 - You can also use directly meshlab (by the way also meshlab libraries have python wrappers)

Open 3D functions for smoothing

- Open3D's current TriangleMesh API includes
 - filter_smooth_simple,
 - filter_smooth_laplacian,
 - filter_smooth_Taubin
 - See smooth_open3d.py
- Not using the cotangent weighting (it uses umbrella operator)
 - Smooth_simple just average neighbors
 - Which method produces the strongest visible smoothing?
 - Which method best preserves the overall shape?
 - What happens when you increase `number\_of\_iterations`?
 - What happens when you increase `lambda\_filter` in Laplacian smoothing?

Other python libraries for mesh processing

- open3D is not the only tool for 3D model processing in python
 - Main focus on reconstruction
 - Limited tools for shape/mesh processing
 - Limited visualization tools
- Many alternatives, some might be better for applications
 - **VTK/Pyvista**
 - Trimesh
 - Pymesh
 - **Libigl**

PyVista



- Po
- curvature_pyvista.py
 - Use **pyvista** functions to estimate curvatures
 - Try to change curvature type
- Exercise: try to reduce the number of triangles or to apply subdivision and recompute the curvature maps.
 - Look at pyvista documentation
 - Subdivide function
 - Decimate function

PyVista

- Powerful high-level Python library for 3D visualization and mesh processing, built on top of VTK (Visualization Toolkit), probably the most powerful free tool for 3D visualization
- Used for CFD, FEM simulation results, medical imaging (CT/MRI volumes), but also point clouds. Visualization first, not good for reconstruction, etc.
- But it supports curvature estimation
 - curvature_pyvista.py
 - Use **pyvista** functions to estimate curvatures
 - Try to change curvature type
 - Exercise: try to simplify or subdivide and recompute the curvature maps.
 - Look at pyvista documentation

Note: pyvista and meshes

- Loading the mesh using pyvista we can easily print the main mesh properties
- PolyData structure
- Check
 - <https://docs.pyvista.org/version/stable/api/>
- We can print basic properties
 - `print(mesh)`
- Or check specific properties, e.g.,
 - `mesh.is_manifold`
 - `mesh.n_open_edges`
- We can estimate mesh quality
 - `compute_cell_quality`

Subdivision/decimation

- **pyvista.PolyDataFilters.subdivide**
- Options butterfly, loop (with smoothing), linear (without smoothing)
- **pyvista.PolyDataFilters.decimate**
 - Quadric mesh decimation
 - There is also another filter
 - **pyvista.PolyDataFilters.decimate_pro**

Code

- Pyvista demo: `smooth_pyvista.py`
- Use Laplacian smoothing, Taubin smoothing



Libigl

- More oriented to geometry computation
- Supports differential operators on meshes and allow the easy demonstration of spectral processing
- <https://libigl.github.io>
 - Gradients, Laplace-Beltrami operator
- C++ library with python bindings
 - <https://libigl.github.io/libigl-python-bindings/tut-chapter0/>

Gradients, laplacian

- Tutorial on <https://libigl.github.io/libigl-python-bindings/tut-chapter1/>
- Curv_igl.ipynb application to our model

Spectral decomposition

- Given the cotangent weight, it can be easily computed with scipy functions
- See spectra.ipynb

Exercise



- Load a model created with a scanning procedure
 - Check model features using pyvista (is manifold? Is all triangle? Face quality?)
 - Compute and display mesh curvatures
 - Apply mesh decimation (factor 2) and save the simplified mesh
 - Apply Laplacian smoothing and Taubin smoothing and compare the result
 - Check mesh quality and volumes after both filterings
 - Advanced: find methods to estimate distance between models
 - Compute spectral decomposition and try to enhance high frequency components

Final homework

- Create a closed mesh from partial SFM reconstructions, using registration and meshing tools seen
- Combine with a part modeled in Blender (e.g., a support for a statue) and create a valid manifold, closed mesh (check)
- Slice to prepare for printing (see next week)
- Upload partial models, final model and G-Code on a zip file
- Deadline: May 10